

Placement Portal Application

Project Report

BS Data Science and Applications – IIT Madras

Name	Ramrup Satpati
Roll Number	24f3004027
Email	24f3004027@ds.study.iitm.ac.in
Programme	BS Data Science and Applications
Institution	Indian Institute of Technology Madras (IIT Madras)

1. Introduction

This report documents the design, development, and implementation of a Placement Portal Application, a comprehensive full-stack web system built to streamline the interaction between students, recruiting companies, and institutional administrators. The application addresses a common challenge in academic institutions: the absence of a unified, efficient, and scalable platform for managing the entire placement lifecycle.

The system enables students to explore job listings and track their applications, allows companies to post positions and manage applicants, and provides administrators with oversight tools to approve users and monitor portal activity. The project reflects practical industry standards through its adoption of RESTful API design, token-based authentication, response caching, and asynchronous background task processing.

2. Technologies Used

2.1 Backend

The backend is built using Python-based technologies and serves as the core of the application's business logic and data management:

- Flask – A lightweight and modular Python web framework used to build and expose REST APIs. Flask's simplicity and extensibility make it well-suited for rapid backend development.

- Flask-SQLAlchemy – An Object-Relational Mapping (ORM) library that abstracts the database layer, enabling seamless model definition and database query execution in Python.
- Flask-JWT-Extended – Provides JSON Web Token (JWT)-based authentication, ensuring that all protected API endpoints are accessible only to authorised users with valid session tokens.
- Flask-CORS – Enables Cross-Origin Resource Sharing (CORS) between the Vue.js frontend and the Flask backend, which operate on separate development ports.
- Flask-Caching with Redis – Implements server-side API response caching using Redis as the caching backend, significantly reducing redundant database queries and improving response times.
- Flask-Mail – Facilitates the dispatch of transactional email notifications, such as application status updates and interview scheduling confirmations.
- Celery with Redis – Manages asynchronous background tasks such as generating and exporting application reports in CSV format, preventing long-running operations from blocking the main request thread.

2.2 Frontend

The frontend provides an interactive and responsive user interface:

- Vue.js – A progressive JavaScript framework used to build a reactive and component-driven user interface. Vue's two-way data binding and component architecture enable dynamic UI updates without full page reloads.
- Vite – A modern frontend build tool and development server that provides fast hot-module replacement (HMR) and optimised production builds.
- Axios – A promise-based HTTP client that handles all API communication between the frontend and the backend, supporting request/response interceptors and error handling.

2.3 Database

- SQLite – A lightweight, file-based relational database used as the primary data store during development. SQLite requires no separate server process and is ideal for rapid prototyping and testing environments.

3. Database Schema Design

3.1 Tables Overview

The database schema is structured around five core tables, each representing a distinct entity within the system:

1. User

The central authentication table that manages all user accounts across roles. It stores the user's name, email address, hashed password, assigned role (student, company, or admin), and an approval flag used by administrators to authorise new company registrations.

2. StudentProfile

Stores academic and professional information specific to student users, including department, CGPA, resume file reference, educational history, technical skills, and prior experience. This table maintains a strict one-to-one relationship with the User table via a foreign key constraint.

3. CompanyProfile

Contains organisational details for registered companies, including the company name, location, and a brief description. Like the StudentProfile, it is linked to the User table through a foreign key, ensuring that each company account corresponds to exactly one user record.

4. Job

Represents individual job postings created by companies. Each job record captures the title, description, required skills, offered salary, required experience, and available benefits. A status field (active or closed) allows companies to control the visibility of their listings.

5. JobApplication

Acts as the central junction table connecting students and job postings, implementing a many-to-many relationship between the StudentProfile and Job tables. Each application record tracks the current status of the application (applied, shortlisted, selected, or rejected), as well as additional metadata such as recruiter feedback, scheduled interview date, and offer letter reference.

3.2 Design Rationale

The schema follows several key design principles:

- Separation of the User and Profile tables ensures that authentication logic remains decoupled from profile-specific data, improving both flexibility and scalability as the system evolves.
- The use of foreign key constraints enforces referential integrity across all related tables, preventing orphaned records and maintaining data consistency.
- The JobApplication table serves as a well-defined junction entity, enabling comprehensive tracking of the entire placement lifecycle from initial application through to offer issuance.
- Status-based fields within the JobApplication table provide a clear and auditable trail of each application's progress, facilitating effective communication between students and companies.

4. API Design

The application follows a RESTful API architecture, in which each endpoint represents a specific resource and is accessed using standard HTTP methods (GET, POST, PUT, DELETE). All API endpoints are protected by JWT authentication, and access is governed by role-based access control (RBAC) to ensure that users can only interact with resources appropriate to their assigned role.

4.1 Authentication Endpoints

- POST /login – Authenticates a user and returns a signed JWT access token for use in subsequent requests.
- POST /req – Registers a new user account with the specified role (student, company, or admin).

4.2 Student Endpoints

- GET /student/jobs – Retrieves all currently active job listings available to students.
- POST /student/apply/<job_id> – Submits a job application for the specified listing on behalf of the authenticated student.
- GET /student/applications – Returns a list of all applications submitted by the authenticated student, along with their current statuses.
- GET | PUT /student/profile – Retrieves or updates the student's academic and professional profile information.

4.3 Company Endpoints

- GET | POST /company/jobs – Retrieves all job listings created by the authenticated company, or creates a new listing.
- GET /company/applicants – Returns a list of all students who have applied to the company's job postings.
- PUT /company/application/... – Allows the company to shortlist, reject, or select applicants, as well as schedule interviews.

4.4 Admin Endpoints

- GET /admin/dashboard – Provides aggregated statistics and an overview of system activity for administrative review.
- PUT /admin/approve/<id> – Approves a pending company registration, granting the company access to the portal.

4.5 Access Control Summary

Role-based restrictions are enforced at the middleware level:

- Students are restricted from accessing company or admin endpoints.
- Companies are restricted from accessing student profile endpoints and admin endpoints.

- Caching is applied selectively to frequently accessed, read-heavy endpoints to improve system performance.

5. System Architecture

The application adopts a three-tier client-server architecture, with a clear separation of responsibilities across three layers: the presentation layer (frontend), the business logic layer (backend), and the data persistence layer (database).

5.1 Component Overview

- Frontend (Vue.js) – Handles all user interface rendering, navigation, and interaction. Communicates with the backend exclusively through Axios HTTP calls.
- Backend (Flask) – Processes all incoming API requests, applies business logic, enforces authentication and authorisation rules, and coordinates data access.
- Database (SQLite) – Persists all structured application data, including user records, profiles, job listings, and application statuses.

5.2 Project Structure

The codebase is organised as follows:

- backend/app.py – Contains all API route definitions and associated request-handling logic.
- frontend/src/views – Houses Vue.js components corresponding to each application view or page.
- models/ – Defines the SQLAlchemy database models representing the application's data schema.

6. Features Implemented

6.1 Core Features

- User Authentication – JWT-based login and session management for all user roles.
- Role-Based Dashboards – Distinct views and functionality tailored to students, companies, and administrators.
- Job Posting and Application System – Companies can post, edit, and close job listings; students can browse and apply.
- Application Status Tracking – Real-time status updates visible to both students and companies throughout the hiring process.
- Profile Management – Students and companies can create and update their respective profile information.

6.2 Advanced Features

- Redis Caching – API response caching reduces database load and improves endpoint response times for frequently accessed data.
- Celery Background Tasks – Long-running operations such as report generation are processed asynchronously, ensuring the API remains responsive.
- CSV Export System – Administrators and companies can export application data as structured CSV reports.
- Email Notifications – Automated emails are dispatched to students upon key lifecycle events, including shortlisting, interview scheduling, and selection.
- Interview Scheduling System – Companies can schedule and communicate interview dates directly through the portal.
- Offer Letter Download – Selected students can access and download their offer letters through the application interface.

6.3 Additional Enhancements

- Dynamic UI Updates – Vue's reactivity system ensures that the interface reflects data changes without requiring full page reloads.
- Optimised API Design – Redundant API calls have been minimised through efficient data fetching and state management.
- Clean Separation of Concerns – A clear boundary between frontend and backend responsibilities supports maintainability and future scalability.

7. Conclusion

This project presents a fully functional, full-stack implementation of a placement management system that demonstrates the practical application of modern web development technologies and architectural best practices. The system integrates secure authentication, role-based access control, asynchronous task processing, API response caching, and automated notifications into a cohesive and scalable application.

The development process reinforced key industry-relevant concepts, including RESTful API design, database normalisation, separation of concerns, and performance optimisation. The resulting system is capable of supporting the core requirements of an institutional placement cell, and its modular architecture allows for straightforward extension and enhancement in future iterations.

8. Declaration of AI Usage

I hereby declare that I have made use of AI-based tools, including ChatGPT, to assist with selected aspects of this project, specifically in the areas of debugging, code optimisation, and documentation formatting. The estimated contribution of AI-assisted support to the overall project is approximately 5 to 10 percent, confined primarily to guidance and textual refinement.

All core implementation work, including system design, architectural decisions, database modelling, API development, and frontend construction, was conceived and executed independently by the undersigned. I accept full responsibility for the correctness, integrity, and originality of the submitted project.

Ramrup Satpati

Roll Number: 24f3004027

BS Data Science and Applications, IIT Madras